限量日志各自证明什么. 排错时最容易把“请求已经发出”当成“数据已经回来”，所以这四行日志的边界要背下来：

- `dwc2: ch6 armed addr=4 ep=1 split=...`：HCSPLT、HCCHAR、HCTSIZ、HCINTMSK(6)、HAINTMSK 和 GINTMSK 都已配置，channel 已使能。仅此而已，键盘的一个字节都还没到。
- `dwc2 ch6 final p=... int=... uf=... issue=... age=... ssack=...`：本轮已经走到“不再需要满足当前 CSPLIT 实时期限”的出口才会打印。
- `usbkbd: channel 6 IRQ received`：HCD 已归还 URB，键盘的 completion 把工作排进了 `usbkbd_wq`。
- `usbkbd: first HID report mod=0 keys=10,...`：workqueue 读到了 DMA 缓冲里的报告，`keys=10` 是 usage `0x10`，即字母 `m`。

`ch6 final` 那一行的字段就是为阶段 8 和 9 准备的：`p` 是本轮处在哪个 split 阶段（`p=1` 表示 complete-split），`int` 是 HCINT，`uf` 是 IRQ 采到的微帧号（HFNUM 低 3 位），`issue` 是本次 channel 使能时的计数器，`age` 是从发出到 IRQ 经过的微帧数，`ssack` 是最近一次 SSPLIT 被 ACK 时的计数器。对比 `issue`、`age` 和 `ssack` 就能分辨“总线慢”还是“软件来晚了”。因为中间状态一律不打印，这行日志还做了限量（前 12 条），空闲键盘的 NAK 轮次也不消耗这个额度。HCINT 常用取值的解码见表 7.3：`0x22` 是 `CHHLTD|ACK`（SSPLIT 已被 TT 接受），`0x42` 是 `CHHLTD|NYET`（CSPLIT 还没有结果），`0x03` 才是 `CHHLTD|XFERCOMPL`（DMA 里是完整报告）。

诊断顺序. 上面四条日志本身就是一条顺序链：停在哪个标记之前，就说明卡在那一层——`armed` 之前是 channel 配置，`armed` 与 `final` 之间是总线和 split 时序，`final` 之后是 workqueue 与 completion。只有 `first HID report` 一直是好的而控制台没有字符时，才需要往下查键码映射、输入子系统和 `/dev/input/eventN`。

从两层轮询到中断驱动. 最早的实现虽然端点类型叫 interrupt-IN，CPU 侧的处理方式仍然是轮询，而且是两层：

```
第一层：ARM Generic Timer IRQ
  -> workqueue_timer_tick()
  -> kbd->poll_work 到期，进入 usbkbd_wq
  -> usbkbd_poll() 主动发起一次 interrupt-IN
第二层：udev->ops->interrupt()
  -> dwc2_usb_interrupt() -> channel_xfer(ch6)
  -> while: rd(HCINT(6))          CPU 忙等本轮完成 / NAK / 出错 / 超时
  -> 解析报告，queue_delayed_work(..., 1) 排下一轮
```

这里要把两件事分开：USB 协议上主机必须发起 IN 事务，这是总线规则；但这些事务由 DWC2 硬件按 `bInterval` 排布、完成后用 IRQ 通知 CPU，还是让 CPU 定时自己启动并忙等，则是实现选择。旧实现选了后者，于是每个 tick 都有一次调度开销，而每次事务又占着一个 CPU 自旋。改成 URB 之后，`keyboard.poll_work` 和 `usbkbd_poll()` 被删掉，取而代之的是每设备的 `kbd->rx_work`（只有 URB completion 会调度它）、host ops 里的 `submit_urb()/kill_urb()`，以及 `HCINTMSK(6)` 打开 XFERCOMPL/CHHLTD/NAK/ACK/NYET 与错误事件。空闲键盘返回 NAK 时 CPU 仍要处理一次很短的 IRQ 和 completion，但等待下一个 `bInterval` 的阶段只保存在 delayed-work 的到期链表里，不再由 CPU 或 kworker 自旋。

现在的执行链把“谁在等”换成了“谁被叫醒”，完整的一条链是这样：

```
usbkbd_probe()
  -> 首次 arm 长期 interrupt-IN DMA 请求，随即返回，不等待传输
DWC2 硬件按 bInterval 调度 IN 令牌
  -> 低速键盘经 LAN951x 的 TT 执行 SSPLIT/CSPLIT
  -> 有报告时 DMA 写入缓冲：HCINT(6).XFERCOMPL -> HAINT -> IRQ 9
dwc2_irq()                          硬中断 / HCD 快速路径
  -> SSPLIT ACK：就地记下 HFNUM，到 issue+2 微帧提交 CSPLIT
  -> 窗口内 NYET：下一个微帧直接重试 CSPLIT
```